

SIMULACIÓN DE TRAYECTORIA LINEAL DE UN ROBOT SCARA EN GAZEBO

Control de posición y análisis de velocidades articulares

ROS 2 Jazzy - Gazebo Sim - MATLAB | 9 de septiembre de 2026

Para esta parte del trabajo se tomó el modelo SCARA que ya estaba definido y se trabajó únicamente la simulación. El objetivo fue hacer que el efector recorriera una línea recta entre dos puntos en un tiempo conocido. Esa línea se dividió en pasos de 0.10 s y, para cada paso, se calculó la posición cartesiana, la posición de cada articulación y la velocidad articular correspondiente. Después, la secuencia de posiciones se ejecutó en Gazebo con `ros2_control` y se verificó el movimiento del robot.

Parámetro	Valor usado
Estructura del robot	SCARA R-R-P
Tiempo total	5 s
Paso de tiempo	0.10 s
Número de puntos	51
Controlador	JointTrajectoryController
Interfaz de comando	Posición
Estados observados	Posición y velocidad
Límite J2	-1.750 a +1.750 rad

1. Configuración de la simulación en Gazebo

La cadena cinemática usada en la simulación fue `base_link -> J1 -> link_1 -> J2 -> link_2 -> J3 -> link_3`. J1 y J2 son articulaciones revolutas y J3 es prismática. En el eje vertical el motor y el piñón permanecen unidos al segundo brazo, mientras que la cremallera es el elemento que realmente sube y baja con J3.

Joint	Tipo	Límite	Movimiento
J1	Revolute	-pi a +pi rad	Giro del primer brazo
J2	Revolute	-1.750 a +1.750 rad	Giro del segundo brazo
J3	Prismatic	-0.084 a +0.084 m	Desplazamiento vertical

El límite de J2 se ajustó después de probar el movimiento en RViz. Con un rango de $\pm 180^\circ$ el segundo brazo alcanzaba a atravesar el primero, por lo que se buscó manualmente el punto seguro antes del contacto y se dejó el rango final en ± 1.750 rad (aprox. $\pm 100.3^\circ$).

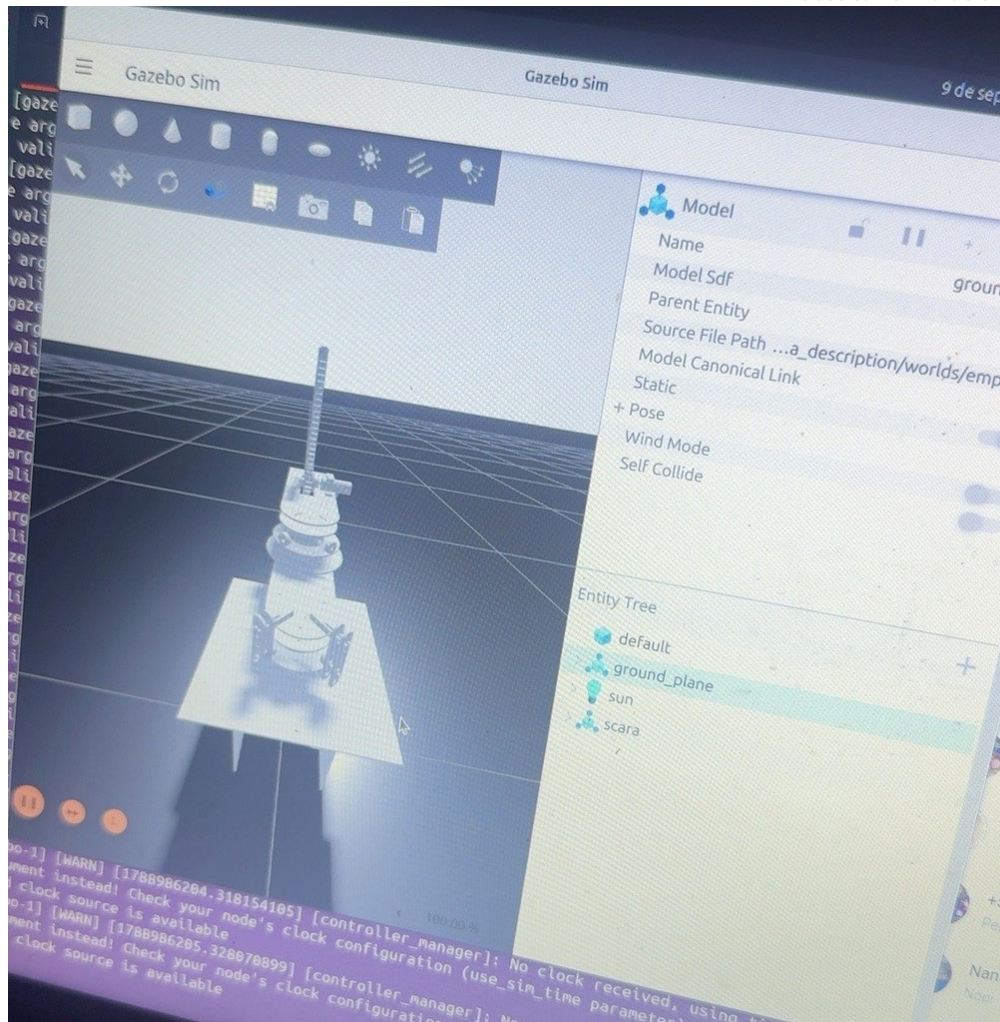


Figura 1. Robot SCARA cargado y visible dentro de Gazebo Sim.

2. Control de posición y lectura de velocidad

El movimiento se hizo con `gz_ros2_control` y un `JointTrajectoryController`. En la configuración final el comando principal es posición. Esto significa que a Gazebo se le entrega la posición que debe alcanzar cada articulación y el instante en el que debe alcanzarla. A partir de esos puntos temporizados el controlador interpola el movimiento. Al mismo tiempo se publican como estados la posición y la velocidad, por lo que se puede revisar cómo se está moviendo cada joint durante la simulación.

```
arm_controller:
  ros__parameters:
    joints: [joint_1, joint_2, joint_3]
    command_interfaces: [position]
    state_interfaces: [position, velocity]
```

Antes de usar los 51 puntos se realizó una prueba simple con $q_1 = 0.5$ rad, $q_2 = 0.5$ rad y $q_3 = 0.05$ m, con 3 s para llegar a la referencia. El objetivo fue aceptado y terminó en estado `SUCCEEDED`, de modo que se confirmó que los tres joints estaban asociados al controlador correcto.

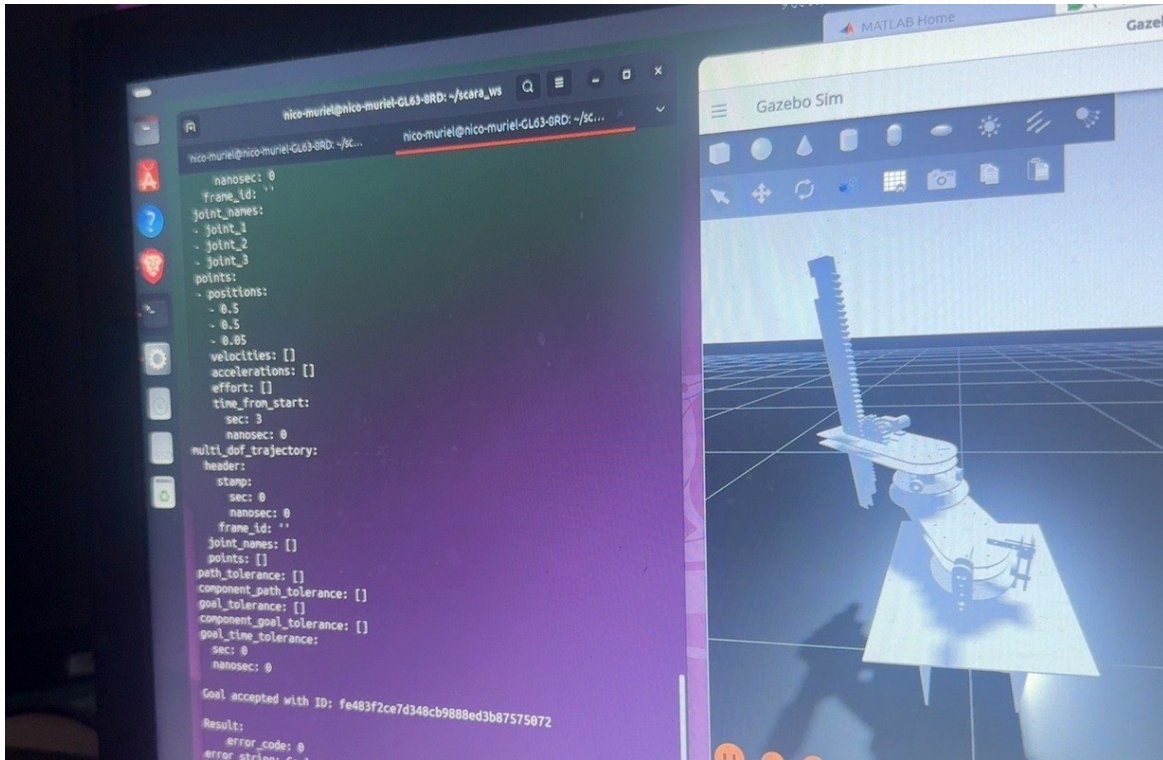


Figura 2. Prueba de control por posición antes de cargar la trayectoria completa.

3. Construcción de la línea recta en MATLAB

La trayectoria se planteó entre dos puntos alcanzables del espacio de trabajo. No se escogieron dos coordenadas al azar: se usaron las longitudes $L1 = 0.15000$ m y $L2 = 0.14617$ m, el límite de J2 y el recorrido vertical de J3 para buscar un segmento recto largo que no obligara al robot a entrar en una configuración no permitida.

Dato	Valor
Punto inicial P0	[-0.2273, 0.1899, 0.3365] m
Punto final Pf	[0.2273, 0.1899, 0.5045] m
Distancia total	0.4847 m
Tiempo total	5.0 s
Velocidad cartesiana	0.0969 m/s
Paso	0.10 s
Cantidad de referencias	51

La distancia entre P0 y Pf fue 0.4847 m. Como se definieron 5 s para completar el recorrido, la magnitud de la velocidad cartesiana fue 0.0969 m/s. Sus componentes fueron aproximadamente $v_x = 0.0909$ m/s, $v_y = 0.0000$ m/s y $v_z = 0.0336$ m/s.

La recta se dividió con $\lambda = t/T$. Para cada tiempo se evaluó $P(t) = P0 + \lambda(Pf-P0)$. Al usar $dt = 0.10$ s aparecen 51 puntos, incluyendo el inicial y el final. Por eso el movimiento no se manda como un solo salto, sino como una sucesión de referencias pequeñas y ordenadas en el tiempo.

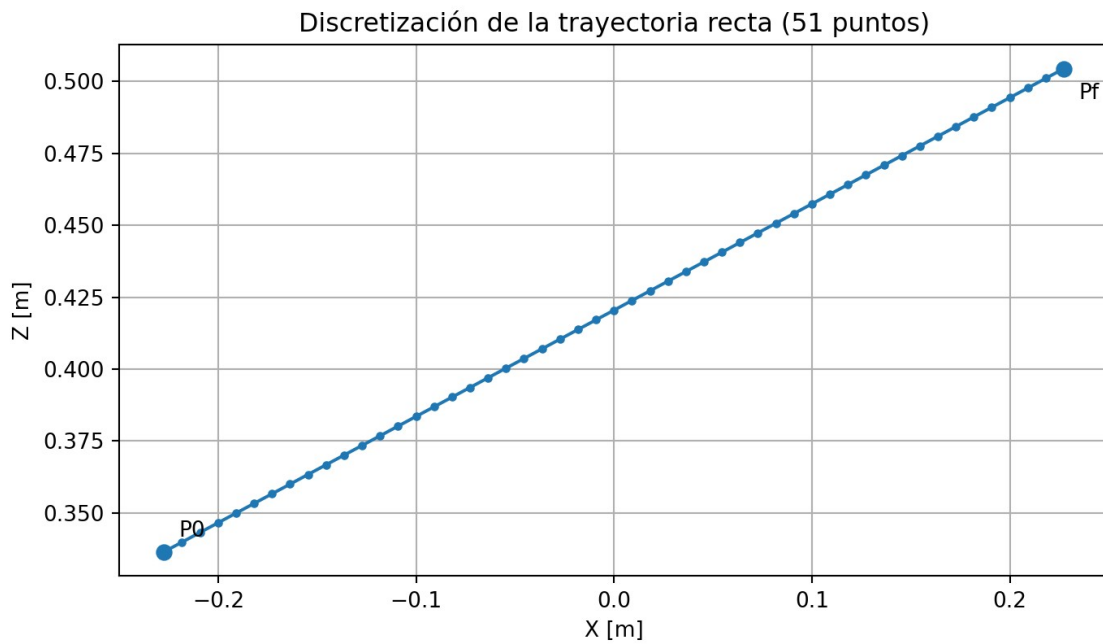


Figura 3. La línea recta discretizada en los 51 puntos que se envían al controlador.

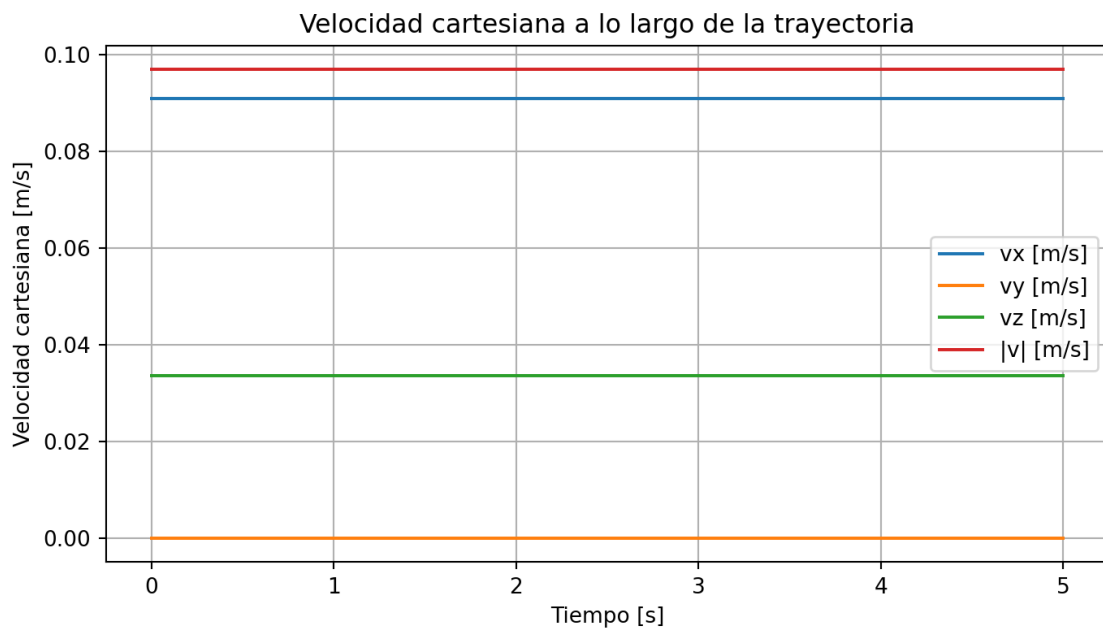


Figura 4. Velocidad cartesiana del recorrido. Al usar interpolación lineal, sus componentes se mantienen constantes.

4. Posición y velocidad en cada paso

En cada uno de los 51 puntos se resolvió la cinemática inversa del SCARA. Primero se conocen X, Y y Z; a partir de ellos se calculan q_1 , q_2 y q_3 . De esta forma cada pedazo de la línea cartesiana queda convertido en una referencia que Gazebo sí puede aplicar a las articulaciones.

```
c2 = (x^2 + y^2 - L1^2 - L2^2)/(2*L1*L2)
q2 = acos(c2)
q1 = atan2(y,x) - atan2(L2*sin(q2), L1 + L2*cos(q2))
q3 = z - z0
```

```

dq1 = gradient(q1,dt)
dq2 = gradient(q2,dt)
dq3 = gradient(q3,dt)

```

Las velocidades articulares se obtuvieron con derivación numérica entre puntos consecutivos. Por eso el CSV no guarda únicamente la posición deseada, sino también dq1, dq2 y dq3 para cada instante. En la ejecución final se enviaron al JointTrajectoryController las posiciones y el tiempo de cada punto; la velocidad queda asociada al cambio de posición entre referencias y se puede comparar con los estados de velocidad publicados por Gazebo.

t [s]	X [m]	Z [m]	q1 [rad]	q2 [rad]	q3 [m]	dq1 [rad/s]	dq2 [rad/s]	dq3 [m/s]
0.000	-0.227	0.336	2.446	0.000	-0.084	-2.339	4.332	0.034
0.500	-0.182	0.353	1.863	0.957	-0.067	-0.706	0.927	0.034
1.000	-0.136	0.370	1.543	1.322	-0.050	-0.602	0.584	0.034
1.500	-0.091	0.387	1.250	1.561	-0.034	-0.573	0.377	0.034
2.000	-0.045	0.404	0.969	1.702	-0.017	-0.545	0.190	0.034
2.500	0.000	0.420	0.711	1.750	0.000	-0.479	0.000	0.034
3.000	0.045	0.437	0.499	1.702	0.017	-0.360	-0.190	0.034
3.500	0.091	0.454	0.357	1.561	0.034	-0.206	-0.377	0.034
4.000	0.136	0.471	0.297	1.322	0.050	-0.030	-0.584	0.034
4.500	0.182	0.488	0.335	0.957	0.067	0.206	-0.927	0.034
5.000	0.227	0.504	0.696	0.000	0.084	1.936	-4.332	0.034

La tabla anterior muestra una referencia cada 0.5 s para que se vea claramente cómo cambian posición y velocidad durante el recorrido. La tabla completa con los 51 pasos se deja al final del informe.

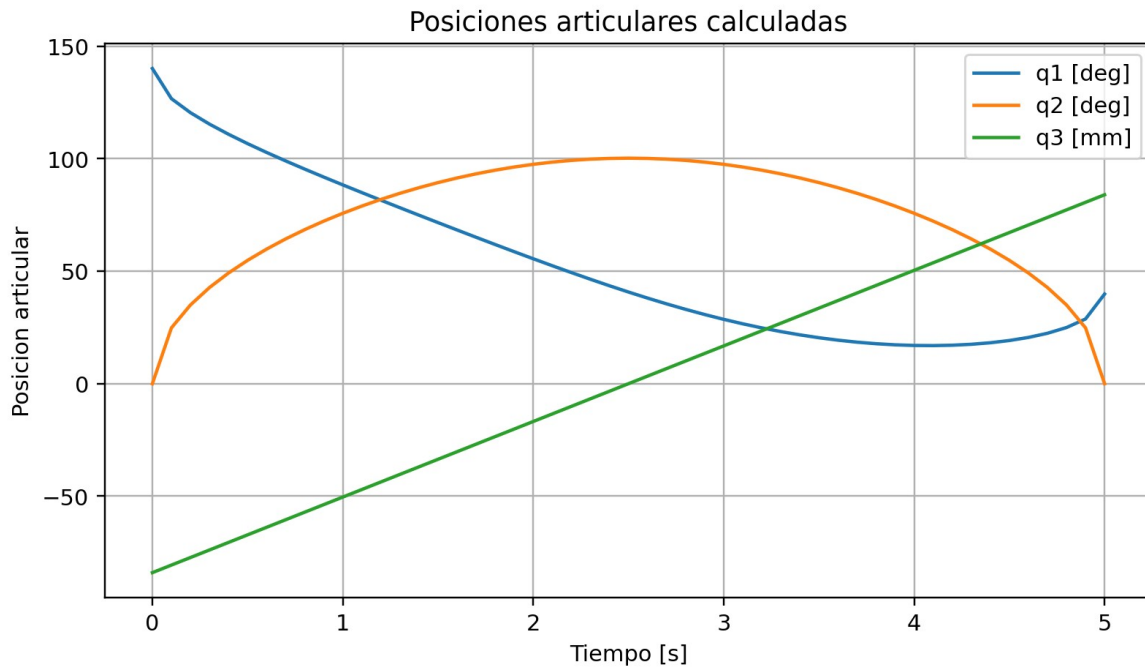


Figura 5. Posiciones articulares calculadas para los 51 puntos.

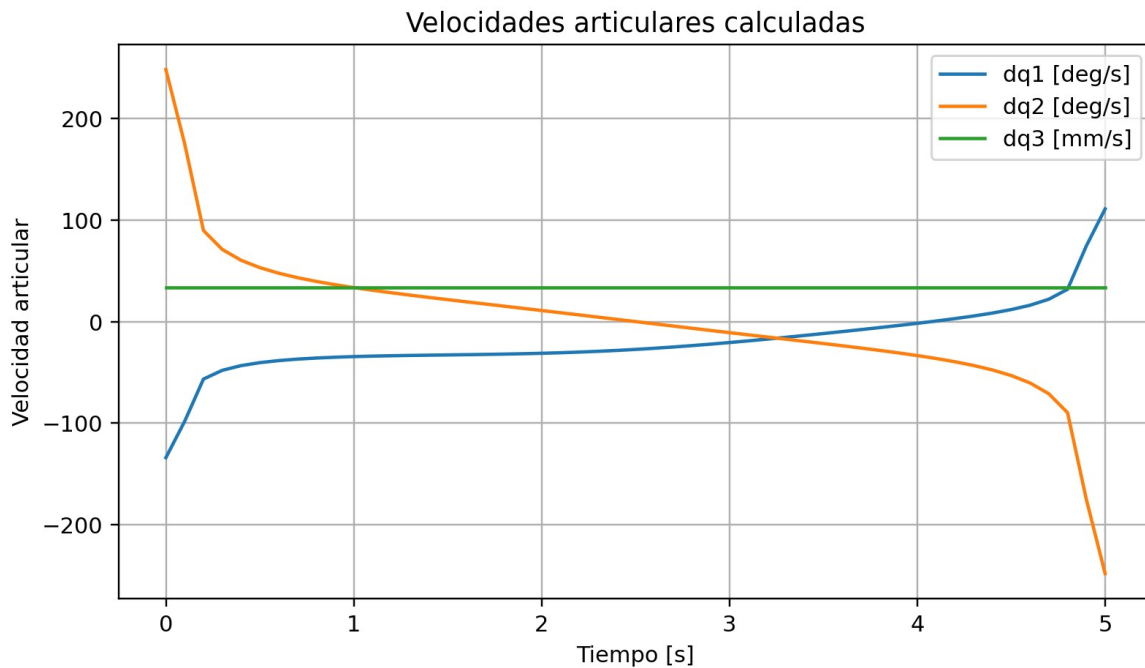


Figura 6. Velocidades articulares asociadas a los puntos de la trayectoria.

En los extremos aparecen picos mayores en $dq1$ y $dq2$ porque la trayectoria empieza y termina cerca de una configuración extendida del mecanismo y las velocidades se calcularon con gradient. Para una puesta en marcha física sería conveniente agregar una ley temporal suave (por ejemplo, perfil trapezoidal o polinomio de quinto orden), pero para esta simulación se mantuvo la condición planteada de recorrer la línea recta a velocidad cartesiana constante.

5. Envío de la trayectoria completa a Gazebo

MATLAB generó `trayectoria_scara.csv` con las 51 filas. El archivo contiene tiempo, X-Y-Z, $q1$ - $q2$ - $q3$ y las tres velocidades articulares. Como MATLAB no estaba instalado en el computador donde se ejecutó Gazebo, el CSV se calculó en otro equipo y luego se copió a la carpeta `config` del paquete ROS 2.

```
ros2 run scara_description send_trajectory.py \
~/scara_ws/src/scara_description/config/trayectoria_scara.csv
```

El primer envío fue rechazado porque se intentaron mandar también las velocidades como comandos dentro de una configuración que estaba trabajando por posición. Se corrigió el emisor para mantener el control por posición y usar el tiempo de cada punto como parte de la referencia. Las velocidades se conservaron en el CSV para el análisis y la comparación con el estado publicado por Gazebo.

En la prueba final la terminal mostró "Enviando 51 puntos", después "Trayectoria ACEPTADA" y al terminar reportó `status = 4`. El resultado correspondió a `SUCCEEDED` y el robot recorrió automáticamente la secuencia completa.

6. Qué se controla y qué se mide

Por eso, para el informe se presentan las dos variables que se pidieron: posición articular en cada punto y velocidad articular en cada punto. Si se requiere guardar los valores entregados por la simulación, el paquete incluye `log_joint_states.py`, que registra posición, velocidad y esfuerzo de `joint_states` mientras se ejecuta la trayectoria.

El torque o la fuerza también se pueden leer como esfuerzo de simulación, pero no se tomaron como resultado principal porque las masas e inercias del URDF son aproximadas. Para que ese dato tenga valor dinámico real sería necesario cargar las propiedades de masa del ensamble de SolidWorks.

Situación	Ajuste realizado
J2 podía chocar con J1	Se fijó el límite en ± 1.750 rad.
El robot aparecía en Entity Tree pero no se veía	Se corrigió GZ_SIM_RESOURCE_PATH para localizar las mallas STL.
ros2 control no estaba disponible	Se instalaron ros-jazzy-ros2-control y ros-jazzy-ros2-controllers.

El envío con velocidades fue rechazado	Se dejó el comando por posición y las velocidades como referencia/estado de análisis.
--	---

8. Resultado

La simulación cumplió con el objetivo planteado. Se definieron dos puntos del espacio de trabajo, se generó entre ellos una trayectoria cartesiana recta de 5 s y se dividió en 51 pasos de 0.10 s. Para cada paso se obtuvo su posición cartesiana, su posición articular y su velocidad articular. Finalmente, las referencias de posición se enviaron a Gazebo mediante `ros2_control` y el robot completó la secuencia con estado `SUCCEEDED`. De esta manera se puede relacionar directamente cada pedazo de la línea recta con las posiciones y velocidades que deben presentar J1, J2 y J3.

Anexo A. Posición y velocidad para cada paso de la trayectoria

La siguiente tabla corresponde directamente al CSV usado en la simulación. Cada fila es un paso de 0.10 s y contiene la posición cartesiana, la referencia articular y la velocidad articular calculada.

t [s]	X [m]	Y [m]	Z [m]	q1 [rad]	q2 [rad]	q3 [m]	dq1 [rad/s]	dq2 [rad/s]	dq3 [m/s]
0.0	-0.227	0.190	0.336	2.446	0.000	-0.084	-2.339	4.332	0.034
0.1	-0.218	0.190	0.340	2.212	0.433	-0.081	-1.714	3.055	0.034
0.2	-0.209	0.190	0.343	2.103	0.611	-0.077	-0.988	1.566	0.034
0.3	-0.200	0.190	0.347	2.014	0.746	-0.074	-0.838	1.240	0.034
0.4	-0.191	0.190	0.350	1.935	0.859	-0.071	-0.757	1.054	0.034
0.5	-0.182	0.190	0.353	1.863	0.957	-0.067	-0.706	0.927	0.034
0.6	-0.173	0.190	0.357	1.794	1.045	-0.064	-0.671	0.831	0.034
0.7	-0.164	0.190	0.360	1.729	1.123	-0.060	-0.645	0.755	0.034
0.8	-0.155	0.190	0.363	1.665	1.195	-0.057	-0.627	0.690	0.034
0.9	-0.145	0.190	0.367	1.603	1.262	-0.054	-0.613	0.634	0.034
1.0	-0.136	0.190	0.370	1.543	1.322	-0.050	-0.602	0.584	0.034
1.1	-0.127	0.190	0.373	1.483	1.378	-0.047	-0.594	0.538	0.034
1.2	-0.118	0.190	0.377	1.424	1.430	-0.044	-0.587	0.495	0.034
1.3	-0.109	0.190	0.380	1.365	1.477	-0.040	-0.582	0.455	0.034
1.4	-0.100	0.190	0.384	1.307	1.521	-0.037	-0.577	0.415	0.034
1.5	-0.091	0.190	0.387	1.250	1.561	-0.034	-0.573	0.377	0.034
1.6	-0.082	0.190	0.390	1.193	1.596	-0.030	-0.569	0.339	0.034
1.7	-0.073	0.190	0.394	1.136	1.628	-0.027	-0.564	0.302	0.034
1.8	-0.064	0.190	0.397	1.080	1.657	-0.024	-0.559	0.265	0.034
1.9	-0.055	0.190	0.400	1.024	1.681	-0.020	-0.553	0.228	0.034
2.0	-0.045	0.190	0.404	0.969	1.702	-0.017	-0.545	0.190	0.034
2.1	-0.036	0.190	0.407	0.915	1.719	-0.013	-0.536	0.152	0.034
2.2	-0.027	0.190	0.410	0.862	1.733	-0.010	-0.524	0.115	0.034
2.3	-0.018	0.190	0.414	0.810	1.742	-0.007	-0.511	0.077	0.034
2.4	-0.009	0.190	0.417	0.760	1.748	-0.003	-0.496	0.038	0.034
2.5	0.000	0.190	0.420	0.711	1.750	0.000	-0.479	0.000	0.034
2.6	0.009	0.190	0.424	0.664	1.748	0.003	-0.459	-0.038	0.034

2.7	0.018	0.190	0.427	0.620	1.742	0.007	-0.437	-0.077	0.034
2.8	0.027	0.190	0.431	0.577	1.733	0.010	-0.413	-0.115	0.034
2.9	0.036	0.190	0.434	0.537	1.719	0.013	-0.388	-0.152	0.034
3.0	0.045	0.190	0.437	0.499	1.702	0.017	-0.360	-0.190	0.034
3.1	0.055	0.190	0.441	0.465	1.681	0.020	-0.332	-0.228	0.034
3.2	0.064	0.190	0.444	0.433	1.657	0.024	-0.302	-0.265	0.034
3.3	0.073	0.190	0.447	0.404	1.628	0.027	-0.271	-0.302	0.034
3.4	0.082	0.190	0.451	0.379	1.596	0.030	-0.239	-0.339	0.034
3.5	0.091	0.190	0.454	0.357	1.561	0.034	-0.206	-0.377	0.034
3.6	0.100	0.190	0.457	0.338	1.521	0.037	-0.172	-0.415	0.034
3.7	0.109	0.190	0.461	0.322	1.477	0.040	-0.138	-0.455	0.034
3.8	0.118	0.190	0.464	0.310	1.430	0.044	-0.103	-0.495	0.034
3.9	0.127	0.190	0.468	0.302	1.378	0.047	-0.067	-0.538	0.034
4.0	0.136	0.190	0.471	0.297	1.322	0.050	-0.030	-0.584	0.034
4.1	0.145	0.190	0.474	0.296	1.262	0.054	0.009	-0.634	0.034
4.2	0.155	0.190	0.478	0.299	1.195	0.057	0.051	-0.690	0.034
4.3	0.164	0.190	0.481	0.306	1.123	0.060	0.096	-0.755	0.034
4.4	0.173	0.190	0.484	0.318	1.045	0.064	0.147	-0.831	0.034
4.5	0.182	0.190	0.488	0.335	0.957	0.067	0.206	-0.927	0.034
4.6	0.191	0.190	0.491	0.359	0.859	0.071	0.281	-1.054	0.034
4.7	0.200	0.190	0.494	0.391	0.746	0.074	0.384	-1.240	0.034
4.8	0.209	0.190	0.498	0.436	0.611	0.077	0.555	-1.566	0.034
4.9	0.218	0.190	0.501	0.502	0.433	0.081	1.301	-3.055	0.034
5.0	0.227	0.190	0.504	0.696	0.000	0.084	1.936	-4.332	0.034